

Conditional Classes & @php Directive

In this lesson, we are going to look at the `@php` directive, conditional classes as well as the `request()` helper function.

@php Directive

The `@php` directive allows you to execute PHP code within your Blade templates. This can be useful when you need to perform some logic that is not possible with the Blade syntax.

Open the `resources/views/components/header.blade.php` file and add this to the very top:

```
@php
    echo 'Hello, World!';
@endphp
```

You should see `Hello, World!` printed at the top of the page.

request() Helper Function

Change the hello world to the following:

```
@php
    $isActive = request()->is('/');
    echo $isActive ? 'active' : '';
@endphp
```

Here, we are using the `request()` helper function to check if the current URL is the homepage. If it is, we set the `$isActive` variable to `true`. We then use a ternary operator to print `active` if `$isActive` is `true`.

You can delete the whole `@php` block for now. I just wanted to show you how to use it.

Conditional Classes

I want to make the active or current link bold and yellow. Right now, the only link that works is the `/jobs` link. Let's add an additional link for the homepage.

```
<a href="{{ url('/') }}" class="text-white hover:underline py-2">Home</a>
```

There are many ways to do conditional classes. We can add a ternary in the class attribute like so:

```
<a href="/" class="{{ request()->is('/') ? 'font-bold text-yellow-500' : '' }}">Home</a>
```

Here are all of the links:

```
<a href="{{ url('/') }}"
  class="text-white hover:underline py-2 {{ request()->is('/') ? 'text-yellow-400 font-bold' : '' }}">Home</a>
<a href="{{ url('/jobs') }}"
  class="text-white hover:underline py-2 {{ request()->is('jobs') ? 'text-yellow-400 font-bold' : '' }}">All
  Jobs</a>
<a href="{{ url('/jobs/saved') }}"
  class="text-white hover:underline py-2 {{ request()->is('jobs/saved') ? 'text-yellow-400 font-bold' : '' }}">Saved
  Jobs</a>
<a href="{{ url('/login') }}"
  class="text-white hover:underline py-2 {{ request()->is('login') ? 'text-yellow-400 font-bold' : '' }}">Login</a>
<a href="{{ url('/register') }}"
  class="text-white hover:underline py-2 {{ request()->is('register') ? 'text-yellow-400 font-bold' : '' }}">Register</a>
<a href="{{ url('/dashboard') }}"
  class="text-white hover:underline py-2 {{ request()->is('dashboard') ? 'text-yellow-400 font-bold' : '' }}">
  <i class="fa fa-gauge mr-1"></i> Dashboard
</a>
```

@class Directive

There is another way that we can do this using a `@class` directive. I am not going to keep this method because of the way we're going to do things in future videos with attributes and props, but I want to show you how to use it.

We can also use the `@class` directive like so:

```
<a href="{{ url('/') }}" @class([
  'text-white hover:underline py-2' ,
  'font-bold text-yellow-400'=>request()->is('/')
])>Home</a>

<a href="{{ url('/jobs') }}" @class([
  'text-white hover:underline py-2' ,
  'font-bold text-yellow-400'=> request()->is('jobs')
])>All Jobs</a>

<a href="{{ url('/jobs/saved') }}" @class([
  'text-white hover:underline py-2' ,
  'font-bold text-yellow-400'=>request()->is('jobs/saved')
])>Saved Jobs</a>

<a href="{{ url('/login') }}" @class([
```

```
'text-white hover:underline py-2' ,  
'font-bold text-yellow-400'=> request()->is('login')  
  
<a href="{{ url('/register') }}" @class([  
    'text-white hover:underline py-2' ,  
    'font-bold text-yellow-400'=>request()->is('register')  
  
<a href="{{ url('/dashboard') }}" @class([  
    'text-white hover:underline py-2' ,  
    'font-bold text-yellow-400'=>request()->is('dashboard')  
    <i class="fa fa-gauge mr-1"></i> Dashboard  
</a>
```

So the `@class` directive takes in an array of classes. The first element is the default class. Then you can add additional classes with the key being the class and the value being the condition.

It is completely up to you on which method you want to use but for this project, we will be using the ternary operator.